Agri-Share | DBMS Viva Guide

Code-level walkthrough of frontend/lib/db.ts

1. CODE-LEVEL EXPLANATION & WALKTHROUGH

Imports: `createClient` constructs a Supabase client; `SupabaseClient` supplies its TypeScript type.

Singleton: `adminClient` begins as `null` and caches the client after first creation. Reuse applies within the current module/runtime instance.

getSupabaseAdmin(): checks whether the cache is empty. If so, it reads `NEXT_PUBLIC_SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` from the server environment.

Configuration guard: if either value is missing, the function throws an explanatory error and stops before client creation.

Client creation: `createClient(url, serviceKey, options)` creates the client. `persistSession: false` disables session persistence and `autoRefreshToken: false` disables automatic token refresh.

Cache and return: the instance is assigned to `adminClient` and returned; later calls reuse it. The final export re-exports the client type.

2. DBMS & ARCHITECTURE CONCEPTS DEMONSTRATED

This file sets up database access; it contains no table definitions or queries. Therefore it does not demonstrate 3NF, keys, referential actions, CHECK constraints, ENUM types, JOINs, or `.from()`, `.select()`, and `.eq()`.

It uses a privileged service-role client. The code comment states this role bypasses Row Level Security (RLS), so trusted server-side routes using it must enforce authentication and authorization. The key must remain server-side and must never be exposed to client components. A browser anon key is intended to operate subject to configured RLS policies.

The lazy singleton avoids repeated client construction within a module instance, but does not imply one shared client across server processes or deployments. This file has no frontend state management or async/await logic.

3. UI & STYLING LOGIC

Not applicable: this is a server-side library module, with no UI components, state management, or styling classes.

4. 30-SECOND PRESENTATION SLIDE SCRIPT

“This file centralizes Agri-Share's privileged Supabase connection for server-side API routes. The getSupabaseAdmin function lazily creates the client the first time it is called, using the project URL and service-role key from environment variables, then reuses that client on later calls. It fails early with a clear error if either setting is missing. Because the service-role key bypasses Row Level Security, this client must stay on the server and be used only in trusted code that applies the appropriate authorization checks.”

5. VIVA DEFENCE Q&A

Q1. Why use a service-role key, and what security consequence follows?
Model answer: It gives trusted server-side code privileged access and bypasses RLS. It must remain secret. Routes using this client need to authenticate and authorize requests before database operations; RLS alone will not protect those operations.

Q2. What does the lazy singleton achieve, and what are its limits?
Model answer: It constructs the client on first call and caches it for later calls in the same module instance, avoiding repeated setup. It is not a universal instance shared across processes, serverless instances, or deployments.

Q3. Why disable session persistence and automatic token refresh?
Model answer: This client uses a privileged service-role key rather than a browser user session. Disabling persistence and refresh means it is not configured as a persistent user session. Authorization must be handled by the server routes that use it.